

Flipkart | Accenture

LC 542: 01 MATRIX

2 Approaches → Multi source BFS, DP

0	0	0
0	1	0
1	1	1



0	0	0
0	1	0
1	2	1

need distance of nearest 0

Approach 1: Multi-Source BFS

→ Imagine aadya is standing on every 0, all zeros start spreading together.

Also make sure that before pushing to queue, you mark them visited to avoid pushing the same 0 back to back to queue.

This works because suppose aadya is 1, 3 zeros start moving towards 1, the first one to reach her must be the nearest.

TC = $O(n \times m)$ **SC = $O(n \times m)$**

C++ Implementation (BFS)

```
class Solution {
public:
    vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {
        int n = mat.size();
        int m = mat[0].size();

        vector<pair<int,int>> dir = {{1,0}, {-1,0}, {0,1}, {0,-1}};
        vector<vector<int>> dist(n, vector<int>(m, 0));
        vector<vector<int>> seen(n, vector<int>(m, 0));
```

```

queue<pair<int,int>> q;

for(int i = 0; i < n; i++){
    for(int j = 0; j < m; j++){
        if(mat[i][j] == 0){
            seen[i][j] = 1;
            q.push({i,j});
        }
    }
}

while(!q.empty()){
    int currsz = q.size();
    while(currsz--){
        int r = q.front().first;
        int c = q.front().second;
        q.pop();

        for(auto it : dir){
            int nr = r + it.first;
            int nc = c + it.second;
            if(nr >= 0 && nc >= 0 && nr < n && nc < m){
                // agar sabse pehle phoche tabhi aadya milegi nhi to koi aur le j
                // and you wont have access to influence aadya as she already found her
                if(!seen[nr][nc]){
                    seen[nr][nc] = 1;
                    dist[nr][nc] = dist[r][c] + 1;
                    q.push({nr,nc});
                }
            }
        }
    }
}

return dist;
}
};

```

Approach 2: Dynamic Programming (DP)

Suppose Aadya is at 1.

Nearest zero can come from $\uparrow \downarrow \leftarrow \rightarrow$

DP cant know all 4 directions simultaneously.

- **pass 1:** Top left $\rightarrow$ bottom right
- **pass 2:** Top left $\leftarrow$ bottom right

For every 1 $\rightarrow$ dist = 1 + min neighbour distance

Step-by-Step Execution

- (S1) initialize the dist matrix with ∞
- (S2) all zeroes have distance = 0
- (S3) top left to bottom right pass which handle: *(Checking top and left neighbors)*
- (S4) bottom right to top left pass which handles: *(Checking bottom and right neighbors)*

C++ Implementation (DP)

```
class Solution {
public:
    vector<vector<int>>> updateMatrix(vector<vector<int>>& mat) {
        int n = mat.size();
        int m = mat[0].size();

        vector<vector<int>>> dp(n, vector<int>(m, INT_MAX - 1));

        // Step S1 & S2
        for(int i = 0; i < n; i++){
            for(int j = 0; j < m; j++){
                if(mat[i][j] == 0){
                    dp[i][j] = 0;
                }
            }
        }

        // Step S3: Top-Left to Bottom-Right pass
        for(int i = 0; i < n; i++){
            for(int j = 0; j < m; j++){
                if(i > 0)
                    dp[i][j] = min(dp[i][j], dp[i-1][j] + 1);

                if(j > 0)
                    dp[i][j] = min(dp[i][j], dp[i][j-1] + 1);
            }
        }

        // Step S4: Bottom-Right to Top-Left pass
        for(int i = n - 1; i >= 0; i--){
            for(int j = m - 1; j >= 0; j--){
                if(i < n - 1)
                    dp[i][j] = min(dp[i][j], dp[i+1][j] + 1);

                if(j < m - 1)
```

```
        dp[i][j] = min(dp[i][j], dp[i][j+1] + 1);
    }
}
return dp;
}
};
```